

## 1. What Spring Boot Is

Spring Boot is an opinionated layer on top of the Spring Framework that lets you build production-ready applications with very little configuration. Instead of wiring every bean by hand, you add a starter dependency and Spring Boot configures sensible defaults for you.

Three ideas make Spring Boot productive:

- Starters: curated dependency sets such as `spring-boot-starter-web` or `spring-boot-starter-data-jpa`. One starter pulls in a consistent, tested set of libraries so you do not have to pick compatible versions yourself.
- Auto-configuration: Spring Boot inspects the classpath and the existing beans, then creates the beans you most likely need. If a `DataSource` driver is on the classpath and you configured a URL, a `DataSource` bean is created. Auto-configuration always backs off when you define your own bean of the same type.
- Embedded server: a web application runs as a plain executable jar with an embedded Tomcat, Jetty or Undertow, so there is no WAR file to deploy.

The `@SpringBootApplication` annotation is a shortcut for three annotations: `@Configuration` (the class can declare beans), `@EnableAutoConfiguration` (turn on auto-configuration) and `@ComponentScan` (scan the package of this class and its sub-packages for components). Place the main class in the root package so that component scanning finds everything.

To see why a bean was or was not auto-configured, start the application with `--debug`. The condition evaluation report lists every auto-configuration class with the conditions that matched or did not match, for example `@ConditionalOnClass` or `@ConditionalOnMissingBean`.

## 2. Inversion of Control and Dependency Injection

Inversion of Control (IoC) means that objects do not create their own dependencies. The Spring container (the ApplicationContext) creates objects, called beans, and hands each bean the collaborators it needs. This hand-over is called Dependency Injection (DI).

Spring supports three styles of injection:

- Constructor injection: dependencies are parameters of the constructor. This is the recommended style. Fields can be final, the object is always fully initialized, required dependencies are obvious, and unit tests can simply call the constructor with mocks.
- Setter injection: useful for optional dependencies that have a reasonable default.
- Field injection with `@Autowired` on a field: short to write but hides dependencies, prevents final fields and makes the class hard to test without a Spring context. Avoid it in production code.

When a class has exactly one constructor, Spring uses it automatically and the `@Autowired` annotation is not required.

```
@Service
public class OrderService {
    private final OrderRepository repository;
    private final PaymentClient paymentClient;

    public OrderService(OrderRepository repository, PaymentClient paymentClient) {
        this.repository = repository;
        this.paymentClient = paymentClient;
    }
}
```

If two beans implement the same interface, injection by type is ambiguous and startup fails with `NoUniqueBeanDefinitionException`. Resolve it with `@Primary` on the default implementation or with `@Qualifier("beanName")` at the injection point.

### 3. Bean Scopes and Lifecycle

The scope of a bean decides how many instances the container creates.

- singleton (default): one shared instance per ApplicationContext. Singleton beans must be thread-safe because many requests use them at the same time; keep them stateless.
- prototype: a new instance every time the bean is requested from the container.
- request, session, application: web-aware scopes, one instance per HTTP request, per HTTP session or per ServletContext.

A classic pitfall is injecting a prototype bean into a singleton. The prototype is created only once, when the singleton is created, so the singleton keeps using the same instance forever. To get a fresh instance each time, inject an `ObjectProvider<MyPrototype>` and call `getObject()`, or use a scoped proxy.

Lifecycle callbacks let a bean run code after injection and before destruction:

- `@PostConstruct` runs after all dependencies are injected. Use it for initialization that needs the dependencies, such as warming up a cache.
- `@PreDestroy` runs when the context is closed. Use it to release resources.

Beans are created eagerly at startup by default, which surfaces configuration errors early. Mark a bean with `@Lazy` to create it only on first use, or set `spring.main.lazy-initialization=true` for the whole application to speed up startup at the cost of discovering errors later.

## 4. Declarative Transactions with @Transactional

@Transactional marks a method (or every public method of a class) as transactional. Spring does not change your code; it wraps the bean in an AOP proxy. When a caller invokes the method through the proxy, the proxy opens or joins a transaction, calls your method, and then commits or rolls back.

Rollback rules are the most misunderstood part:

- By default the transaction rolls back on unchecked exceptions (RuntimeException and its subclasses) and on Error.
- A checked exception (for example IOException or a custom exception extending Exception) does NOT trigger rollback by default. The transaction commits even though the method threw.
- Use rollbackFor = Exception.class to roll back on checked exceptions too, or noRollbackFor to keep committing for a specific runtime exception.

```
@Transactional(rollbackFor = Exception.class)
public void importFile(Path file) throws IOException { ... }
```

readOnly = true is a hint for read-only work. With Hibernate it disables dirty checking and flushing for that transaction, which saves memory and CPU on large reads, and some drivers route it to a read replica.

Other attributes: timeout (seconds before the transaction is rolled back), isolation (for example Isolation.REPEATABLE\_READ) and propagation, which is explained on the next page.

In Spring Boot, a PlatformTransactionManager is auto-configured (JpaTransactionManager when JPA is present), so @Transactional works without any extra setup.

## 5. Transaction Propagation: REQUIRED vs REQUIRES\_NEW

Propagation defines what happens when a transactional method is called while a transaction may already be running.

**REQUIRED** (the default): support a current transaction, create a new one if none exists. If the caller already has a transaction, the called method joins it; both methods share one physical transaction and commit or roll back together. If the inner method throws a runtime exception, the whole shared transaction is marked rollback-only.

**REQUIRES\_NEW**: always create a new, independent transaction. If a transaction is already active, it is suspended until the new transaction finishes. The inner transaction commits or rolls back on its own. Consequently, if a **REQUIRES\_NEW** method commits successfully, its data stays saved even if the outer transaction rolls back afterwards. A typical use case is writing an audit log or an error record that must survive a failed business operation.

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void writeAuditLog(String action) { auditRepository.save(new AuditLog(action)); }
```

**REQUIRES\_NEW** needs a second database connection while the outer transaction is suspended, so heavy use can exhaust the connection pool.

Other propagation types:

- **NESTED**: runs inside the current transaction using a savepoint; the inner part can roll back to the savepoint without rolling back the outer transaction.
- **MANDATORY**: requires an existing transaction, otherwise throws an exception.
- **SUPPORTS**: joins a transaction if there is one, otherwise runs without a transaction.
- **NOT\_SUPPORTED**: suspends the current transaction and runs without one.
- **NEVER**: throws an exception if a transaction exists.

## 6. Proxy Pitfalls: Self-Invocation and Rollback-Only

Because `@Transactional` is implemented with a proxy, the annotation only takes effect when the call goes through the proxy.

Self-invocation: when a method calls another method of the same class (`this.otherMethod()`), the call does not pass through the proxy. The `@Transactional` settings of `otherMethod` are silently ignored, including `REQUIRES_NEW`. The same applies to `@Async` and `@Cacheable`.

```
public void placeOrder() {  
    saveOrder();  
    this.writeAuditLog();    // proxy bypassed: no new transaction!  
}
```

Fixes: move the method to a separate bean and inject that bean, or restructure the code so that the caller is another bean. Injecting the bean into itself also works but is harder to read.

Visibility: with the default proxy mode, annotating private methods has no effect. Annotate public methods.

`UnexpectedRollbackException`: when an inner method with `REQUIRED` throws a runtime exception, the shared transaction is marked rollback-only. If the outer method catches that exception and tries to finish normally, the commit fails with `UnexpectedRollbackException`:

"Transaction silently rolled back because it has been marked as rollback-only". Either let the exception propagate, or use `REQUIRES_NEW` for the inner call if its failure must not affect the outer work.

## 7. Spring Data JPA Repositories

Spring Data JPA removes most data-access boilerplate. You declare an interface and Spring generates the implementation at runtime.

```
public interface BookRepository extends JpaRepository<Book, Long> {  
    List<Book> findByAuthorAndPublishedYearGreaterThan(String author, int year);  
    Optional<Book> findByIsbn(String isbn);  
}
```

JpaRepository provides save, findById, findAll, deleteById, count and paging support out of the box.

Derived query methods: Spring parses the method name (findBy, countBy, existsBy, deleteBy followed by property names and keywords such as And, Or, Between, Like, In, OrderBy) and builds the JPQL query. Invalid names fail at startup, which catches typos early.

@Query lets you write JPQL (or native SQL with nativeQuery = true) when the name would become too long. Modifying queries need @Modifying and a transaction.

Pagination: add a Pageable parameter and return Page<T> or Slice<T>. Page runs an extra count query to know the total number of elements; Slice only knows whether a next slice exists, so it is cheaper for infinite scrolling.

```
Page<Book> page = repository.findAll(PageRequest.of(0, 20, Sort.by("title")));
```

Projections: return an interface or a record with only the needed fields to avoid loading whole entities.

save() on an entity with an assigned identifier first checks whether the entity is new. If the id is set manually and there is no @Version field, Spring Data calls merge, which executes an extra SELECT before the INSERT.

## 8. Externalized Configuration and Profiles

Spring Boot reads configuration from many sources: application.properties or application.yml, profile-specific files, environment variables, command-line arguments and more. Sources later in the precedence order override earlier ones; for example, an environment variable overrides a value in application.yml, and a command-line argument overrides both. This is how the same jar runs with different settings in each environment.

Relaxed binding maps different spellings to the same property: app.rag.top-k in YAML, APP\_RAG\_TOPK as an environment variable and appRag.topK all bind to the same field.

Profiles group configuration per environment. application-dev.yml is loaded only when the dev profile is active, set with spring.profiles.active=dev or the SPRING\_PROFILES\_ACTIVE environment variable. Beans can be limited to a profile with @Profile("dev").

@Value("\${app.timeout}") injects a single value. For groups of related settings prefer @ConfigurationProperties: it binds a whole prefix to a class or record, supports validation with @Validated, gives IDE auto-completion through generated metadata, and keeps configuration type-safe.

```
@ConfigurationProperties("app.mail")
public record MailProperties(String host, int port, Duration timeout) {}
```

Enable it with @ConfigurationPropertiesScan or @EnableConfigurationProperties. Never commit secrets such as passwords or API keys to the repository; read them from environment variables or a secret manager.

## 9. REST Controllers and Validation

`@RestController` combines `@Controller` and `@ResponseBody`: return values are serialized to JSON by Jackson instead of being resolved as view names.

Mapping annotations: `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping` and `@DeleteMapping`. Method parameters are bound with `@PathVariable` (part of the URL path), `@RequestParam` (query string or form field), `@RequestBody` (JSON body) and `@RequestHeader`.

Return `ResponseEntity` when you need to control the status code or headers, for example 201 Created with a Location header after creating a resource.

Bean Validation (Jakarta Validation) checks input declaratively. Put constraints on the DTO and add `@Valid` in front of the `@RequestBody` parameter:

```
public record CreateUserRequest(
    @NotBlank @Size(max = 50) String name,
    @Email String email,
    @Min(18) int age) {}

@PostMapping("/users")
ResponseEntity<UserDto> create(@Valid @RequestBody CreateUserRequest req) { ... }
```

When validation fails, Spring throws `MethodArgumentNotValidException` and responds with 400 Bad Request. To validate `@RequestParam` or `@PathVariable` values, annotate the controller class with `@Validated`; violations then raise `ConstraintViolationException` (or `HandlerMethodValidationException` in newer versions).

Keep controllers thin: they translate HTTP to method calls and back. Business rules belong in the service layer, which is easier to test and reuse.

## 10. Error Handling with ProblemDetail

A consistent error format makes an API easier to consume. Spring Framework 6 supports RFC 9457 Problem Details (which replaced RFC 7807). A problem response has the media type `application/problem+json` and the fields `type`, `title`, `status`, `detail` and `instance`, plus any custom properties.

Enable the built-in support with `spring.mvc.problemdetails.enabled=true`. Spring MVC exceptions, such as validation errors or an unsupported media type, are then rendered as `ProblemDetail` automatically.

Centralize the handling of your own exceptions in a class annotated with `@RestControllerAdvice`. Each `@ExceptionHandler` method handles one exception type:

```
@RestControllerAdvice
class GlobalExceptionHandler extends ResponseEntityExceptionHandler {
    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail handle(OrderNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Order not found");
        pd.setProperty("orderId", ex.getOrderId());
        return pd;
    }
}
```

Extending `ResponseEntityExceptionHandler` reuses Spring's handling for standard MVC exceptions. Alternatively, custom exceptions can extend `ErrorResponseException`, which already carries a `ProblemDetail` body and status.

Do not leak stack traces or internal messages to clients; log the details on the server and return a friendly message.

## 11. Testing Spring Boot Applications

Spring Boot provides test slices that start only part of the application, which keeps tests fast.

- `@SpringBootTest` starts the full application context. Use it for integration tests; combine with `webEnvironment = RANDOM_PORT` to test through real HTTP.
- `@WebMvcTest(MyController.class)` loads only the web layer (controllers, advice, converters) and provides `MockMvc`. Services are replaced with `@MockitoBean` mocks.
- `@DataJpaTest` loads only JPA components (entities, repositories) and wraps each test in a transaction that is rolled back at the end.

`Testcontainers` starts real dependencies, such as PostgreSQL, in Docker for the duration of the tests. With `@ServiceConnection`, Spring Boot reads the container's host, port and credentials automatically, so no manual property wiring is needed:

```
@Bean
@ServiceConnection
PostgreSQLContainer postgres() {
    return new PostgreSQLContainer("postgres:16");
}
```

Testing against the same database engine as production catches problems that an in-memory database such as H2 would hide, for example SQL dialect differences or database-specific types.

Spring Boot Actuator adds production endpoints: `/actuator/health` for liveness and readiness probes, `/actuator/metrics` for Micrometer metrics, and `/actuator/info`. Expose only what you need with `management.endpoints.web.exposure.include`.